

Traffic Signs Detection in Road Scenes

By Nisioi Mihai-Florin, Oprean Cristian, Simedroni Andra

1. Abstract

This work presents a real-time traffic sign detection and recognition pipeline designed for road scenes captured under both daylight and nocturnal conditions. The system performs adaptive image segmentation via multi-thresholding, guided by an environmental selector that classifies frames as “day” or “night” using mean intensity statistics. Candidate blobs are extracted from a binary representation and filtered by realistic geometric constraints (size, area, aspect ratio). To achieve rotation- and scale-invariant matching, each candidate’s inner region is mapped to log-polar coordinates and evaluated against reference shapes using normalized cross-correlation (NCC). Features from accepted candidates are reduced with Neighborhood Preserving Embedding (NPE) and classified using a Support Vector Machine (SVM). The entire application is implemented in Python.

2. State of the Art

Traffic sign pipelines span color/shape heuristics and hand-crafted features to deep CNNs. Classical color-space thresholding and geometry cues remain common but struggle across illumination and scale; recent CNN variants (multi-scale and committee models) push accuracy to $\approx 98.31\text{--}99.46\%$ on GTSRB, at the cost of heavy hyperparameter tuning and long training times, which limits deployability.

Our target paper advances a hybrid alternative: environment selection (day/night) with multi-thresholding to extract robust inner regions, followed by log-polar + NCC for rotation/scale-tolerant shape validation, and low-dimensional subspace features classified by SVM—achieving $\approx 98.3\%$ recognition with strong precision/recall under varied day/night sequences.

Compared to CNN committees, this design reports far lower training cost (dozens of features, minutes vs. ≈ 50 h) while remaining competitive on GTSRB and robust to adverse night/rain scenes.

3.3. Theoretical Fundamentals

3.1. Segmentation

We first select the operating regime (day/night) using the frame’s global mean intensity: values in the 0–40 range indicate night; larger values indicate day. This stabilizes the subsequent color processing under changing illumination. For daytime, the image is mapped to the red-angle plane to emphasize the red-rimmed signs:

$$\text{Angle}(R) = \frac{R}{\sqrt{R^2 + G^2 + B^2}}(1)$$

For night-time, we enhance white interiors against dark backgrounds using morphological top-hat (ITI_TIT) and bottom-hat (IBI_BIB) operations on the preprocessed image IPI_PIP, and form the enhanced grayscale which highlights the bright inner region of the sign. We then apply multiple thresholding to obtain seven binary maps per regime. For day images, thresholds follow:

$$M = \mu + k\sigma, k \in \{1 \dots 7\} \quad (2)$$

on the red-angle plane (typical 0.54–0.87). For night images, evenly spaced thresholds in 70–250 are used on:

$$I_{EG} = (I_P + I_B) - I_T \quad (3)$$

After binarization, the inner region of a sign appears as a black hole (day) or solid white (night), which reduces false negatives and shrinks the search space for detection.

3.2. Detection

Detection is realised by scanning the binary image to find the connected components that might be traffic signs. Connected components (blobs) are extracted from each binary map and filtered by geometric plausibility:

- height and width in [16, 128] px
- area ≥ 250 px
- aspect ratio in [0.5, 2.1]

To make detection independent of rotation and scale, we convert each blob's inner region to log-polar coordinates:

$$\rho = \log \sqrt{(x - x_0)^2 + (y - y_0)^2} \quad (4)$$

We then validate the shape by Normalized Cross-Correlation (NCC) against reference templates; candidates with correlation ≥ 0.7 are accepted as valid signs and the rest are discarded. This log-polar + NCC test is robust to global gain/bias and avoids part-based or sliding-window complexity.

3.3. Recognition

For each shape-validated ROI, the pictogram is recognized in a low-dimensional, locality-preserving subspace. Log-polar images are projected using Neighborhood Preserving Embedding (NPE) obtained from a graph of nearby samples; solving the generalized eigenproblem yields a linear projection $y = Z^T \cdot x$ that preserves local class structure with ~ 30 features per sample. A one-vs-all SVM with Gaussian (RBF) kernel then assigns the class label

$$f(x) = \text{sgn} \left(\sum_{i=1}^{N_s} a_i l_i e^{-||s_i - x||^2 / (2\sigma)} + b \right) \quad (5)$$

This NPE+SVM stage provides compact, discriminative features with few

hyperparameters, complementing the earlier segmentation and detection to deliver accurate classification of the detected signs.

4. Implementation Details

4.1 Processing the image

We defined a global `TRESHOLD_MEAN`, of default value 40. After converting the image into grayscale, we compute the average value of all pixels in the image (illumination level), and compare it with the threshold in order to classify the image as daytime/nighttime.

```
img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
global_mean_intensity = np.mean(img)

# Use the threshold for automatic classification
if global_mean_intensity > THRESHOLD_MEAN:
    classification = "Daytime (AUTO)"
else:
    classification = "Nighttime (AUTO)"

return classification, global_mean_intensity
```

The next function implements the Red-Angle transformation, computing the „dominant” red component and comparing it to overall color intensity of a pixel. We first convert the 8-bit integers into floats and separate the image into its three individual color channels (Red, Green, Blue). Afterwards, we apply the formula in the documentation and we obtain the grayscale image where bright pixels indicate strong red content, while dark pixels indicate non-red areas.

```
def calculate_red_angle(img_rgb):
    """Calculates the Red Angle image plane for daytime segmentation."""
    img_rgb = img_rgb.astype(np.float32)
    R, G, B = img_rgb[:, :, 0], img_rgb[:, :, 1], img_rgb[:, :, 2]
    denom = np.sqrt(R**2 + G**2 + B**2) + 1e-6
    RedAngle = R / denom
    return RedAngle
```

If the image is classified as **daytime**, we compute 7 binary masks, in order to ensure that at least one threshold will perfectly capture the sign's shape. These threshold values lay between 0.54 and 0.87, so we clip these values to stay within the useful range. The last step is to implement the binary function $I(T)$, where pixels satisfying the condition become white, and the others black.

```
for k in range(1, 8):
    M = Mu + k * Sigma
    M_clipped = np.clip(M, 0.54, SLIDER_CLIP_MAX)
    BinaryImage = (RedAngle > M_clipped).astype(np.uint8) * 255
    OutBinaryImg.append((BinaryImage, f"Threshold k={k}"))
    log_messages += f"k={k}, Threshold M = {M_clipped:.4f}\n"
```

If the image is classified as **nighttime**, we apply Enhanced Grayscale Transform. First, we convert the image to grayscale. Then, we compute the value of the kernel, by dynamically computing the size of the kernel (1% of the smallest image dimension). The next step is to apply morphological filtering (Top-Hat & Bottom-Hat). Top-Hat highlights small bright objects on a dark background and bottom-hat highlights small dark regions inside bright areas. Afterwards, we compute I_{EG} , emphasize the inner white pixels of the traffic sign. The final step is to use multiple thresholding, to overcome lighting conditions problem and to minimize false negatives.

```
gray = cv2.cvtColor(InputImage_RGB, cv2.COLOR_RGB2GRAY)

h, w = gray.shape
base_size = min(h, w)
kernel_size = max(3, int(base_size * 0.01))
if kernel_size % 2 == 0:
    kernel_size += 1

kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (kernel_size, kernel_size))
log_messages += f"Adaptive kernel size: {kernel_size}x{kernel_size}\n"

top_hat = cv2.morphologyEx(gray, cv2.MORPH_TOPHAT, kernel)
bottom_hat = cv2.morphologyEx(gray, cv2.MORPH_BLACKHAT, kernel)

IEG = cv2.add(gray, top_hat)
IEG = cv2.subtract(IEG, bottom_hat)

IEG = cv2.normalize(IEG, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
IntermediateTitle = f"Enhanced Grayscale (IEG) | Kernel {kernel_size}x{kernel_size}"
IntermediateImage_RGB = cv2.cvtColor(IEG, cv2.COLOR_GRAY2RGB)

thresholds = [70, 100, 130, 160, 190, 220, 250]
for t in thresholds:
    _, BinaryImage = cv2.threshold(IEG, t, 255, cv2.THRESH_BINARY)
    OutBinaryImg.append((BinaryImage, f"Threshold t={t}"))
    log_messages += f"Threshold = {t}\n"
```

The paper proposes using thresholds between 70 and 250, but we added a slider in the interface in order to find the best threshold for the night images, as there are some in which the flashlights are occupying a significant amount of the image, and it applies red-angle for nighttime.

```
threshold_slider = tk.Scale(
    slider_container, from_=0, to=100, orient=tk.HORIZONTAL, resolution=1,
    command=update_threshold,
    length=150, bg=BACKGROUND_COLOR, fg=TEXT_COLOR,
    troughcolor=DISPLAY_FRAME_COLOR, highlightthickness=0,
    borderwidth=1, activebackground=BUTTON_COLOR,
    showvalue=0
)
threshold_slider.set(THRESHOLD_MEAN)
threshold_slider.grid(row=1, column=0, padx=(0, 5), sticky="w")
```

4.2 Implementation of the GUI

The interface is built using Tkinter, the layout being split into a control panel (for the user's input) and a main content area, in which we display the images and data.

As Tkinter cannot display raw OpenCv image, we created a function that converts the raw image data into a Python Imaging Library, realizing it to fit the UI.

```
def resize_and_convert(img_numpy, target_width, target_height):
    """Resizes a numpy image (RGB/BGR) and converts it to a Tkinter PhotoImage."""
    img_pil = Image.fromarray(img_numpy)

    ratio = min(target_width / img_pil.width, target_height / img_pil.height)
    new_width = int(img_pil.width * ratio)
    new_height = int(img_pil.height * ratio)

    resized_img = img_pil.resize((new_width, new_height), Image.Resampling.LANCZOS)
    return ImageTk.PhotoImage(resized_img), new_width, new_height
```

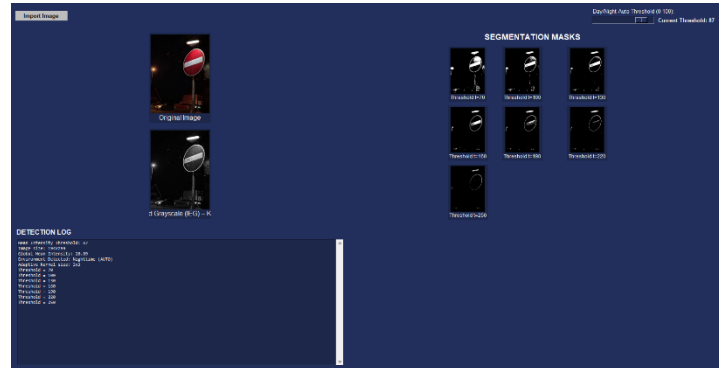
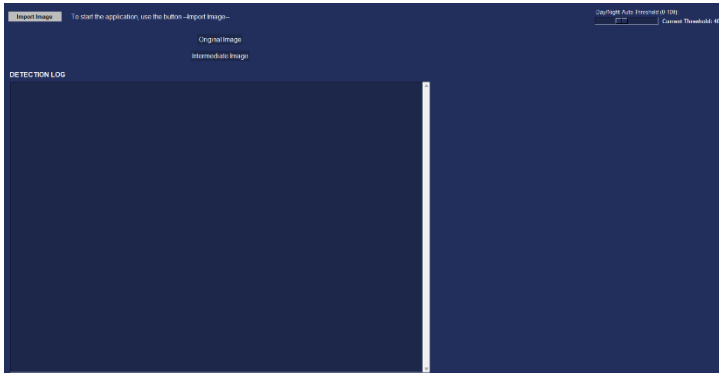
The 3x3 grid of images that display multiple thresholding are implemented with the help of the function `clear_mask_grid_frame()`:

```
def clear_mask_grid_frame():
    """Destroys all previous widgets in the mask display area."""
    for widget in mask_grid_frame.winfo_children():
        widget.destroy()
    global mask_tk_images
    mask_tk_images = []
```

The interface is dynamic, as it connects the Slider and Import Button to the processing logic. When the slider is moved, it updates the global variable `THRESHOLD_MEAN`.

Regarding the GUI itself, it features three main views:

- **Original/Intermediate Image:** Shows the input and the **preprocessing** result (e.g., Red-Angle Plane or Grayscale).
- **Segmentation Masks:** Displays the binary outputs resulting from different **intensity thresholds**.
- **Detection Log:** Records parameters, including the **Environment Detection** (Daytime/Nighttime).



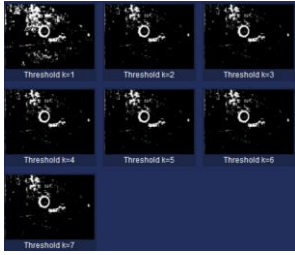
5. Experimental Results and Conclusions

This chapter presents the experimental segmentation results across five distinct test cases, highlighting the algorithm's performance under various conditions, including different illumination levels (Daytime vs. Nighttime) and sign types (circular vs. triangular).

5.1. Daytime results

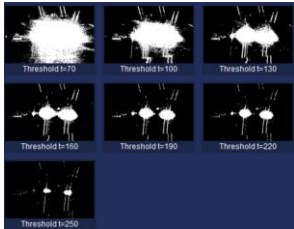
The algorithm demonstrates strong and robust performance for traffic signs captured under daytime or clear overcast conditions. The use of the **Red-Angle Image Plane** is consistently effective in isolating the target features:

CASE	INTERFACE RESULT	SIGN TYPE	IMG. PLANE USED	OPTIMAL MASK	KEY CONCLUSION
Case 1		Speed limit 50 (Circle)	Red-Angle	k=1 to k=7	Highly Robust: Segmentation of the circular boundary is stable across all tested thresholds.
Case 2		Yield (Triangle)	Red-Angle	k=1 to k=7	Geometric Accuracy: Successfully isolates the unique inverted triangle shape with perfect clarity.

Case 3		Speed limit 100 (Circle)	Red- Angle	k=1 to k=7	Clean Extraction: Achieves a clean isolation of the sign from a busy highway background due to high contrast provided by the Red-Angle plane.
--------	---	--------------------------------	---------------	---------------	---

5.3. Night-time results

The algorithm's performance degrades under challenging nighttime and low-light conditions, where it becomes significantly more **sensitive to the exact threshold value** selected. For these cases, an **Iced Grayscale** or similar preprocessing is used instead of the Red-Angle Plane.

CASE	INTERFACE RESULT	SIGN TYPE	IMG. PLANE USED	OPTIMAL MASK	KEY CONCLUSION
Case 1		Speed limit 100 (Circle)	Iced Grayscale (EG)	Threshold b = 70	Threshold Sensitivity: Performance is highly unstable; higher thresholds (e.g. b=250) cause complete data loss.
Case 2		No Entry (Triangle)	Iced Grayscale (EG)	Threshold b = 70	Narrow Range: The segmentation is highly effective only in a very narrow, low-end range of the threshold (b=70), quickly vanishing thereafter.

6. Bibliography

- [1] Alsibai M., and Hirai Y. "Real-time recognition of blue traffic signs designating directions." Int J Intell Transp Syst Res, vol. 8, no. 2, 2010, pp. 96–105.
- [2] Chang CC, and Lin CJ. "LIBSVM: a library for support vector machines." ACM Trans Intell Syst Technol, vol. 2, no. 27, 2011, <http://www.csie.ntu.edu.tw/~cjlin/libsvm>.
- [3] Duda RO, et al. Pattern classification. 2 ed., New York, NY, Wiley-Interscience, 2000.
- [4] Fleyeh H., and Davami E. "Eigen-based traffic sign recognition." IET Intelligent Transport Systems, vol. 5, no. 3, 2011, pp. 190-196.
- [5] H, Liu, et al. Traffic sign recognition using group sparse coding. vol. 266, Inform Sci, 2014 .
- [6] Zaklouta, F., and F. Stanciulescu. Real-time traffic sign recognition in three stages. vol. 62, Robot Auton Syst, 2014. 1 vols.